Concurrency & Distributed Systems

Week 2 — Part 4: Race Conditions

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

May 11, 2026

Block 4 — The Intentional Bug

Today we intentionally break correctness.

Goal

Understand why correct coordination does NOT guarantee correct data.

What Is a Race Condition?

Definition

A race condition occurs when the outcome depends on uncontrolled thread timing.

Real-world example: Two users booking the *last seat* on a flight $\rightarrow$ both succeed $\rightarrow$ overbooking. Requires:

- Shared mutable state
- At least one write
- No synchronization protecting it

Different interleavings $\Rightarrow$ different results.

The Hidden Problem: `count++`

Looks Simple

```
count++
```

Actually expands to:

1. Read count
2. Add 1
3. Write result

Note: at the JVM level this becomes multiple operations (e.g., load / add / store) $\Rightarrow$ not atomic.

This is NOT atomic.

Visualizing the Lost Update

Initial value: `count = 5`

T0: A reads 5 | B reads 5

T1: A writes 6 | B writes 6

T2: Final value = **6** (should be 7)

Lost update

Key idea

Two correct-looking increments can produce one incorrect final result.

Two Threads — One Variable

Initial value: `count = 5`

- Thread A reads 5

Result

One increment is lost.

Two Threads — One Variable

Initial value: `count = 5`

- Thread A reads 5
- Thread B reads 5

Result

One increment is lost.

Two Threads — One Variable

Initial value: `count = 5`

- Thread A reads 5
- Thread B reads 5
- Thread A writes 6

Result

One increment is lost.

Two Threads — One Variable

Initial value: `count = 5`

- Thread A reads 5
- Thread B reads 5
- Thread A writes 6
- Thread B writes 6

Result

One increment is lost.

Common Misconception

Misconception

"If I use `join()`, my result must be correct."

Reality

`join()` guarantees completion and visibility, **not** mutual exclusion or atomicity.

Intentional Race — Coding Task

```
class Counter {
        static int count = 0;
}
Runnable task = () -> {
        for (int i = 0; i < 10000; i++) {
                Counter.count++;}
};
Thread[] threads = new Thread[10];
for (int i = 0; i < 10; i++) {
        threads[i] = new Thread(task);
        threads[i].start();
}
for (int i = 0; i < 10; i++) {
        threads[i].join();
}
System.out.println("Expected: 100000");
System.out.println("Actual: " + Counter.count);
```

Expected vs Actual

Expected

100000

Actual

Varies every run:

- 97342
- 98901
- 99512

! Silent Corruption

No crash. No exception. Just wrong.

Important Observation

- All threads completed (`join()` used)
- Memory visibility guaranteed
- Yet result is incorrect

Critical Insight

Completion does NOT imply atomicity.

Why `join()` Did Not Save Us

`join()` guarantees:

- Completion ordering
- Memory visibility (*happens-before*)

`join()` does NOT guarantee:

- Mutual exclusion
- Atomic operations

$$\text{Coordination} \neq \text{Protection}$$

Recall: `join()` enforces visibility, not safety.

Debugging Prompt

Insert temporary print statements:

- Print thread name
- Print value before increment

What you will observe: multiple threads read the same value, then overwrite each other.

Overwrite Pattern (Example)

Thread	Reads	Writes
A	5	6
B	5	6

Reality

This is silent corruption.

Formal Conditions for Race

A race condition exists when:

1. Two or more threads access the same variable

Rule

If **any one condition is missing**, the race cannot occur.

- No shared state $\rightarrow$ thread-local variables
- No writes $\rightarrow$ read-only configuration
- With synchronization $\rightarrow$ `synchronized`, `AtomicInteger`, locks, etc.

Formal Conditions for Race

A race condition exists when:

1. Two or more threads access the same variable
2. At least one access is a write

Rule

If **any one condition is missing**, the race cannot occur.

- No shared state $\rightarrow$ thread-local variables
- No writes $\rightarrow$ read-only configuration
- With synchronization $\rightarrow$ `synchronized`, `AtomicInteger`, locks, etc.

Formal Conditions for Race

A race condition exists when:

1. Two or more threads access the same variable
2. At least one access is a write
3. There is no synchronization

Rule

If **any one condition is missing**, the race cannot occur.

- No shared state $\rightarrow$ thread-local variables
- No writes $\rightarrow$ read-only configuration
- With synchronization $\rightarrow$ `synchronized`, `AtomicInteger`, locks, etc.

Preview: Fixing It (Before / After)

Before (Unsafe)

```
Counter.count++; // not atomic
```

After (Safe)

```
// Option A: synchronized
synchronized(lock) {
                Counter.count++;
        }
// Option B: AtomicInteger
        AtomicInteger c = new AtomicInteger
            ();
        c.incrementAndGet();
```

Full solution next lecture.

Thread Dump Lab

Run:

Terminal

jps
jstack -l <PID> > dump.txt

Example snippet:

```
"Contender" #16 BLOCKED
waiting to lock <0x000000076b>
```

Legend

- **BLOCKED**: trying to enter a `synchronized` region but cannot
- **waiting to lock**: wants a monitor it does not own
- `<0x...>`: identifier for the monitor object

Guided Dump Analysis

- Which thread is BLOCKED?
- On what object (monitor)?
- Who owns the monitor?
- How many RUNNABLE threads exist?

Connect to:

- Lifecycle states
- Monitor ownership

Micro Quiz

1. Can a TERMINATED thread restart?
2. Does `sleep()` release locks?
3. Is RUNNABLE equal to executing?
4. Does priority guarantee order?
5. Is `count++` atomic?

Block 4 Summary

- Race conditions cause silent corruption
- `join()` ensures completion, not safety
- `count++` is not atomic
- Shared mutable state requires synchronization

Key Insight

Concurrency errors are often invisible.

One Rule to Remember

If multiple threads write shared data,
you MUST synchronize access.

The Uncomfortable Truth

We now know:

- Threads can interleave unpredictably
- `count++` is not atomic
- `join()` ensures completion, not correctness
- Race conditions cause silent corruption

The Problem

If multiple threads write shared data, correctness is not guaranteed.

The Key Question

How do we guarantee that
only one thread modifies shared data at a time?

What We Need

- Mutual exclusion
- Controlled access
- Deterministic behavior

From Chaos to Control

We introduced:

- Race conditions
- Lost updates
- Interleaving problems

Next step:

Synchronization
Locks
Critical Sections

Week 3 Preview — Synchronization & Locks

In Week 3 we will:

- Eliminate race conditions
- Understand intrinsic locks
- Protect critical sections
- Create and detect deadlocks
- Inspect threads using tools

Goal

Move from unpredictable behavior to controlled concurrency.

One Final Thought

Concurrency is not dangerous.
Uncontrolled concurrency is.

Week 3 — We take control.